

A Roots-of-Unity Interpretation of Wheel Sieving and Pollard $p - 1$

Albert Meyburgh

Draft manuscript, May 2026

Abstract

Wheel factorization removes integers lying in residue classes divisible by a fixed set of small primes. This note gives an equivalent formulation in terms of primitive roots of unity: if M is a squarefree wheel modulus, the admissible residues modulo M are exactly the exponents that generate primitive M -th roots of unity. We formalize this equivalence, show that a natural “root uniqueness” collision test reduces exactly to divisibility by previously selected primes, and derive an exact Ramanujan-sum representation of the wheel mask. We then use the same vocabulary to describe Pollard’s $p - 1$ factorization method, where a smooth exponent schedule can force a collapse to the identity modulo one hidden prime component but not modulo the full composite modulus. The contribution is explanatory and structural. It is not a new primality test, not a new factorization algorithm, and not an asymptotically faster prime generator. Its performant form is the conventional one: cached coprime residue classes, cyclic gap tables, small-prime filters, smooth-order factorization attempts, and separate probable-prime or proof-producing stages.

Keywords. prime generation; integer factorization; wheel sieve; roots of unity; primitive roots of unity; Ramanujan sums; Pollard $p - 1$; Pocklington certificates; primality proving.

Scope of claims. This paper does not introduce a new primality test, a new factorization algorithm, or an asymptotically faster prime generator. It gives a roots-of-unity interpretation of existing wheel sieving and smooth-order factorization ideas. Implementations should evaluate this interpretation as ordinary modular arithmetic and precomputed residue/gap tables, not as complex roots, floating-point angles, or rational-angle searches.

1 Introduction

Roots of unity expose a basic distinction between prime and composite orders. For prime p , every nontrivial p -th root of unity is primitive, while for composite n the nontrivial n -th roots split among lower-order cyclic structures. This observation suggests a prime-generation heuristic: search for integers whose associated root-of-unity exponents display maximal novelty or primitivity.

The purpose of this note is to make that heuristic precise. The central result is that, when the idea is applied to candidate sieving, it is exactly wheel factorization in a spectral disguise. Let

$$M = \prod_{p \in B} p$$

be the product of a finite set B of small primes. A conventional wheel keeps precisely those residues $r \pmod{M}$ for which $\gcd(r, M) = 1$. If $\zeta_M = e^{2\pi i/M}$, these residues are precisely the exponents r for which ζ_M^r is a primitive M -th root of unity. Thus the wheel is the primitive-exponent set of the cyclic group of M -th roots of unity.

This reformulation has two effects. First, it provides a concise algebraic and spectral interpretation of a classical algorithmic device. Second, it clarifies what kind of computational gain is possible. Directly manipulating complex roots or exact rational angles gives no advantage; once simplified, the method is the same coprimality filter used in ordinary wheels. The practical version is therefore to precompute residue gaps and use those gaps to generate arbitrary-precision candidates, followed by Miller–Rabin, Baillie–PSW, Pocklington, ECPP, or another primality test/proof method.

1.1 Contributions

This note makes six modest contributions.

1. It states and proves the exact equivalence between wheel survivors and primitive root-of-unity exponents.
2. It records the corresponding negative consequence: any deterministic filter that depends only on whether ζ_M^n is primitive is exactly a wheel mask modulo M .
3. It formalizes a root-collision criterion and proves that the criterion is exactly divisibility by a previously selected prime.
4. It derives an exact Ramanujan-sum representation of the wheel mask, making the spectral content explicit.
5. It gives a roots-of-unity interpretation of Pollard’s $p - 1$ factorization method as a smooth-order root-collision search modulo hidden prime factors.
6. It separates the mathematical reformulation from the computational consequence: the spectral view is conceptually useful, while the performant implementation remains a cached wheel, smooth-order factorization attempts, and independent primality testing or proving.

2 Background and notation

For $M \geq 1$, write

$$(\mathbb{Z}/M\mathbb{Z})^\times = \{r \pmod{M} : \gcd(r, M) = 1\}.$$

Let $\zeta_M = e^{2\pi i/M}$. The order of ζ_M^r is

$$\text{ord}(\zeta_M^r) = \frac{M}{\gcd(r, M)}.$$

Thus ζ_M^r is primitive if and only if $r \in (\mathbb{Z}/M\mathbb{Z})^\times$.

A wheel sieve chooses a modulus M , usually a product of small primes, and keeps the periodic set of residues coprime to M . Pritchard’s wheel sieve gives a classical dynamic form of this idea, and later work studied both imperative and functional wheel enumerators [11, 12, 14]. The present note does not modify those algorithms. It changes the representation: admissible wheel residues are treated as primitive-root exponents.

Remark 1 (Terminology). The phrase “primitive root of unity” is used here in the complex or cyclic-group sense: ζ_M^r has exact order M . This should not be confused with a primitive root modulo a prime, which is a generator of the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^\times$. The two notions share order language, but they occur in different groups.

We also use the Ramanujan sum

$$c_q(n) = \sum_{\substack{1 \leq a \leq q \\ \gcd(a, q) = 1}} e^{2\pi i a n / q},$$

which is the sum of the n -th powers of the primitive q -th roots of unity. Ramanujan sums have a large literature beginning with Ramanujan's work on trigonometric sums [13]. The basic identities used here are elementary.

3 Wheel residues as primitive root exponents

Definition 1 (Wheel mask). Let $M \geq 1$. Define

$$W_M(n) = \begin{cases} 1, & \gcd(n, M) = 1, \\ 0, & \gcd(n, M) > 1. \end{cases}$$

The set of admissible wheel residues modulo M is

$$\mathcal{U}_M = \{r \in \{0, 1, \dots, M-1\} : W_M(r) = 1\}.$$

Theorem 1 (Primitive-exponent equivalence). *For every integer n and modulus $M \geq 1$,*

$$W_M(n) = 1 \iff \zeta_M^n \text{ is a primitive } M\text{-th root of unity.}$$

Proof. The order of ζ_M^n is $M / \gcd(n, M)$. This order equals M exactly when $\gcd(n, M) = 1$. $\square$

Corollary 1 (Prime-base wheel). *Let B be a finite set of primes and $M = \prod_{p \in B} p$. Then*

$$W_M(n) = 1 \iff p \nmid n \text{ for every } p \in B.$$

Equivalently, the wheel keeps exactly the exponents that generate primitive M -th roots of unity.

Proof. Since M is squarefree and has prime divisors exactly B , $\gcd(n, M) = 1$ if and only if none of the primes in B divides n . The primitive-exponent statement follows from the theorem. $\square$

Corollary 2 (No extra filtering information). *Fix $M \geq 1$. Any deterministic accept/reject rule that depends on n only through the truth value of “ ζ_M^n is primitive” is a function only of $W_M(n)$. In particular, the primitive-root condition supplies exactly the wheel mask modulo M and no additional information about divisibility by primes outside M .*

Proof. By the primitive-exponent equivalence, the truth value of “ ζ_M^n is primitive” is identical to $W_M(n)$. Therefore any deterministic rule depending only on that truth value factors through the wheel mask. $\square$

This is the root-of-unity interpretation of an ordinary wheel. For example, for $M = 30$, the admissible residues are

$$1, 7, 11, 13, 17, 19, 23, 29,$$

which are exactly the exponents r for which ζ_{30}^r is primitive.

4 The root-collision criterion

A natural implementation of the root-of-unity idea stores rational angles a/p associated with prime denominators p and rejects a candidate n if some angle r/n has been seen before. The following lemma shows exactly what this test computes.

Lemma 1 (Collision equals divisibility). *Let p be prime, $n > 1$, $1 \leq s < p$, and $1 \leq r < n$. Then*

$$\frac{r}{n} = \frac{s}{p}$$

for some r, s if and only if $p \mid n$.

Proof. If $r/n = s/p$, then $rp = sn$. Since p is prime and $1 \leq s < p$, we have $\gcd(s, p) = 1$. Hence $p \mid n$. Conversely, if $p \mid n$, choose $r = n/p$ and $s = 1$. Then $r/n = 1/p$. $\square$

Proposition 1 (Stored prime-denominator angles implement trial division). *Let P be a finite set of stored primes. A candidate n has a collision with some stored angle whose denominator is in P if and only if*

$$\gcd\left(n, \prod_{p \in P} p\right) > 1.$$

Proof. Apply the lemma independently to each $p \in P$. $\square$

Thus the exact rational-angle version of the collision test is neither false nor mysterious: it is a wheel or trial-division filter. Floating-point angles should not be used, because numerical rounding can introduce false collisions or miss exact ones. Exact rational arithmetic is correct but slower than direct modular arithmetic, and direct modular arithmetic is slower than a precomputed wheel-gap stream when candidates are generated sequentially.

5 A Ramanujan-sum wheel mask

The previous sections used primitive roots only as a set. Ramanujan sums provide a compact spectral expression for the same filter.

For a prime p ,

$$c_p(n) = \sum_{a=1}^{p-1} e^{2\pi i a n / p}.$$

The geometric-series identity gives

$$c_p(n) = \begin{cases} p-1, & p \mid n, \\ -1, & p \nmid n. \end{cases}$$

Therefore

$$D_p(n) = \frac{c_p(n) + 1}{p}$$

is the exact indicator of divisibility by p .

Theorem 2 (Root-of-unity product mask). *Let $M = \prod_{p \in B} p$ be squarefree. Then*

$$W_M(n) = \prod_{p \in B} \left(1 - \frac{c_p(n) + 1}{p}\right).$$

Proof. The factor $(c_p(n) + 1)/p$ is 1 exactly when $p \mid n$ and 0 otherwise. Hence each factor in the product is 1 exactly when $p \nmid n$ and 0 otherwise. The product is 1 exactly when no prime in B divides n , which is precisely $W_M(n) = 1$. $\square$

There is also a divisor-sum form.

Theorem 3 (Ramanujan expansion of a squarefree wheel). *Let M be squarefree. Then*

$$W_M(n) = \frac{\varphi(M)}{M} \sum_{d \mid M} \frac{\mu(d)}{\varphi(d)} c_d(n),$$

where μ is the Moebius function and φ is Euler's totient.

Proof. Both sides are multiplicative as functions of the squarefree modulus M . It therefore suffices to check $M = p$ prime. In that case the right-hand side is

$$\frac{p-1}{p} \left(1 - \frac{c_p(n)}{p-1} \right).$$

If $p \mid n$, then $c_p(n) = p-1$ and the expression is 0. If $p \nmid n$, then $c_p(n) = -1$ and the expression is 1. This agrees with $W_p(n)$, and multiplicativity gives the result for squarefree M . $\square$

The expansion shows that the wheel is a finite Fourier object supported on primitive-root sums. It is exact, periodic modulo M , and entirely equivalent to the coprimality mask.

6 Computational consequences

6.1 Candidate density

For a prime-base wheel $M = \prod_{p \in B} p$, the surviving density among all integers is

$$\frac{\varphi(M)}{M} = \prod_{p \in B} \left(1 - \frac{1}{p} \right).$$

If the candidate generator already restricts to odd integers and $2 \in B$, then the surviving fraction among odd candidates is

$$2 \frac{\varphi(M)}{M}.$$

Base primes	M	$\varphi(M)$	Kept among all integers	Kept among odd integers
2, 3, 5	30	8	0.2667	0.5333
2, 3, 5, 7	210	48	0.2286	0.4571
2, 3, 5, 7, 11	2310	480	0.2078	0.4156
2, 3, 5, 7, 11, 13	30030	5760	0.1918	0.3836
2, 3, 5, 7, 11, 13, 17	510510	92160	0.1805	0.3611

These are useful constant-factor reductions. They are not asymptotic improvements over wheel sieving, because they are wheel sieving.

6.2 Efficient implementation

The performant implementation does not evaluate roots of unity. It builds the primitive-exponent set once and then uses cyclic gaps.

```
BuildWheel(B):
  M <- product of primes in B
  R <- [r in {1,...,M-1} such that gcd(r,M)=1]
  G <- cyclic gaps between consecutive elements of R
  return (M, R, G)

GenerateCandidates(start, M, R, G):
  choose the first n >= start with n mod M in R
  i <- index of n mod M in R
  loop:
    yield n
    n <- n + G[i]
    i <- (i + 1) mod length(G)
```

For arbitrary-precision candidates, n may be a multiprecision integer while M and the gaps remain native machine integers. This is important: a wheel of size 30030 or 510510 often captures most of the practical small-prime filtering benefit while keeping the table modest. Enlarging M reduces candidate density but increases memory, setup cost, and cache pressure.

For random large-prime search, a typical production pipeline is

random odd candidate $\rightarrow$ wheel/small-prime filter
 $\rightarrow$ probable-prime test
 $\rightarrow$ proof or certificate if required.

This is consistent with common multiprecision libraries, which combine small-factor filtering with probable-prime tests such as Miller–Rabin [7].

6.3 Why explicit roots do not improve speed

The spectral masks above are finite periodic functions of $n \bmod M$. Once M is fixed, all information in the root-of-unity formulation is contained in the bitset

$$\{W_M(0), W_M(1), \dots, W_M(M-1)\}$$

and in its gap sequence. Computing complex exponentials, storing floating-point angles, or comparing rational fractions reconstructs this same periodic mask at higher cost. The useful computational step is to precompute the mask and generate only admissible residues.

This does not diminish the mathematical formulation. It says that the formulation is best used to understand and organize the sieve, not to evaluate the sieve literally.

6.4 Benchmarking protocol

This draft intentionally does not report performance measurements without a reproducible benchmark harness. A useful implementation paper should report at least the following comparisons over fixed hardware, fixed random seeds, and published input sets.

Pipeline	Candidate sizes	Primary metrics	Notes
Odd-only generator	256, 512, 1024, 2048 bits	candidates/sec; tests/sec	baseline
Wheel 30, 210, 2310	same	candidates/sec; rejection rate	small tables
Wheel 30030, 510510	same	candidates/sec; cache effects	practical upper range
Wheel plus Miller–Rabin	same	time-to-probable-prime	probable-prime output
Wheel plus proof layer	selected sizes	time-to-certificate	only if implemented
External baseline	same	relative speed	e.g. GMP-style workflow

The important distinction is between mathematical density and measured throughput. Larger wheels reduce the density of candidates reaching primality testing, but they can also increase setup time, memory use, and cache pressure. Benchmark tables should therefore include both rejection-rate data and wall-clock throughput rather than reporting density alone.

7 Root collisions in Pollard $p - 1$ factorization

The same roots-of-unity language also describes a useful factorization mechanism. A wheel filter rejects an integer when its residue is nonprimitive modulo a visible modulus M . Pollard’s $p - 1$ method instead searches for a nontrivial root collapse modulo an unknown prime factor of a composite integer n [9, 6].

Let

$$M_B = \text{lcm}(1, 2, \dots, B) = \prod_{\ell \leq B} \ell^{\lfloor \log_{\ell} B \rfloor}.$$

The prime powers in this product form a reusable exponent schedule. Applying the schedule incrementally to a base a computes $a^{M_B} \bmod n$ without explicitly constructing the enormous exponent M_B .

Proposition 2 (Smooth-order root collision). *Let n be composite and let $p \mid n$ be a prime factor. Suppose $p - 1 \mid M_B$ and $p \nmid a$. Then*

$$a^{M_B} \equiv 1 \pmod{p}.$$

If, in addition, $a^{M_B} \not\equiv 1 \pmod{n}$, then

$$1 < \gcd(a^{M_B} - 1, n) < n,$$

so a nontrivial factor of n is exposed.

Proof. Since p is prime and $p \nmid a$, Fermat’s theorem gives $a^{p-1} \equiv 1 \pmod{p}$. Because $p - 1 \mid M_B$, it follows that $a^{M_B} \equiv 1 \pmod{p}$. Thus p divides $a^{M_B} - 1$, so the greatest common divisor $g = \gcd(a^{M_B} - 1, n)$ is divisible by p and is greater than 1. The additional assumption $a^{M_B} \not\equiv 1 \pmod{n}$ implies that $n \nmid a^{M_B} - 1$, hence $g < n$. $\square$

In root-of-unity terms, the exponent schedule forces a to become the identity root of unity in the component group modulo p . If the same collapse does not occur simultaneously in every component of $(\mathbb{Z}/n\mathbb{Z})^\times$, the mismatch is visible as a nontrivial greatest common divisor. This is a collision between the componentwise cyclic structures hidden inside the composite modulus.

A stage-1 implementation can be written as follows.

```

BuildPMinusOneSchedule(B):
  S <- []
  for each prime ell <= B:
    e <- largest power ell^k <= B
    append e to S
  return S

PollardPMinusOneRootCollision(n, S, bases):
  for a in bases:
    x <- a mod n
    if 1 < gcd(x,n) < n: return gcd(x,n)
    for e in S:
      x <- x^e mod n
      g <- gcd(x-1,n)
      if 1 < g < n: return g
  return failure

```

The schedule S is independent of n and can be cached with metadata such as the bound, entry count, and a content hash. This gives a direct computational meaning to the phrase “root schedule”: the same prime-power exponent list can be reused against many composite inputs.

A standard extension is stage 2. After stage 1 computes $x = a^{M_{B_1}} \bmod n$, one tests primes q with $B_1 < q \leq B_2$ by accumulating

$$\prod_{B_1 < q \leq B_2} (x^q - 1) \pmod{n}$$

and taking a greatest common divisor with n . This can reveal factors for which

$$p - 1 = sq,$$

where $s \mid M_{B_1}$ and q is a single larger prime in the stage-2 range. The root-collision interpretation is unchanged: the larger prime completes the order collapse modulo one hidden factor.

This section does not claim a new factorization algorithm. Pollard’s method is classical. The point is that the same primitive-root vocabulary used to describe wheel masks also describes smooth-order factorization: both are attempts to detect nontrivial structure by forcing or excluding roots of unity in finite cyclic components.

8 Connection to primality proving

The root-of-unity viewpoint becomes more than a filter in primality proving. Pocklington’s theorem uses a large known factor of $n - 1$ to certify that a candidate n is prime. In a simple form, suppose q is prime, $q \mid n - 1$, and $q > \sqrt{n}$. If there exists an integer a such that

$$a^{n-1} \equiv 1 \pmod{n}$$

and

$$\gcd\left(a^{(n-1)/q} - 1, n\right) = 1,$$

then n is prime [10, 4].

Conceptually, this says that the modular multiplicative structure contains an element whose order has a large q -component. That is a modular analogue of detecting a sufficiently primitive

root-of-unity component. Here the root-of-unity perspective is not merely a wheel mask: it helps explain why a witness certifies the absence of small prime factors.

A practical prime generator can exploit this by constructing candidates of the form

$$n = 2kq + 1,$$

where q is already proven prime and $q > \sqrt{n}$. After wheel filtering and probable-prime testing, the Pocklington witness gives a certificate. For arbitrary numbers not built with a known large factor of $n - 1$, general primality-proving methods such as ECPP are more appropriate [2, 8].

Pocklington and ECPP should be treated as proof layers, not as consequences of the wheel mask itself. If an implementation only performs wheel filtering, trial division, Pollard splitting, and Miller–Rabin-style probable-prime testing, it should describe its output as probable-prime or factored output rather than as a proof-producing prime generator. Certificate-producing claims should be made only when certificate construction and verification are implemented.

9 Relation to cyclotomic primality tests

The present construction should not be confused with cyclotomic primality tests, although it shares the same mathematical vocabulary. The AKS test proves primality through polynomial congruences modulo $X^r - 1$ [1]. Jacobi-sum based algorithms, including the Adleman–Pomerance–Rumely line developed by Cohen and Lenstra, use cyclotomic fields and roots of unity more deeply [5]. Those algorithms are primality tests or proofs. The wheel construction here is a candidate filter.

The distinction is important. A filter removes many composites cheaply but cannot certify primality. A primality test or proof determines primality of a candidate. Roots of unity occur in both settings, but with different computational roles.

10 Conclusion

Prime candidate sieving by primitive roots of unity is mathematically coherent. The exact statement is simple: a number survives a wheel modulo M if and only if its exponent indexes a primitive M -th root of unity. A root-collision implementation is exactly a divisibility test by stored primes. Ramanujan sums provide an exact spectral expression of the same mask.

The same language also describes Pollard $p - 1$: a cached smooth exponent schedule can force an element to collapse to the identity root of unity modulo one hidden prime factor of a composite modulus. When the collapse is not simultaneous modulo all factors, a greatest common divisor exposes a nontrivial factor.

The computational conclusion is conservative. The reformulation gives no independent asymptotic improvement over wheel factorization or Pollard $p - 1$, and literal evaluation of complex roots or rational angles is slower than modular arithmetic. Its useful implementation is a cached residue/gap wheel, cached prime-power root schedules, small-prime filtering, smooth-order factorization attempts, and standard probable-prime tests; when proof is required, Pocklington or ECPP remains the appropriate layer.

The main value of the approach is therefore structural. It connects wheel sieves, primitive roots of unity, Ramanujan sums, smooth-order factorization, and primality certificates in a single framework. This may be useful pedagogically, as an implementation design model, and as a starting point for further work on batch spectral masks or certificate-oriented prime construction. Empirical performance claims should be added only with reproducible benchmark scripts and published input sets.

Acknowledgment

The author thanks the open-source and computational number theory communities whose sieve, primality testing, and primality proving tools inform the implementation perspective taken here.

References

- [1] M. Agrawal, N. Kayal, and N. Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004. <https://annals.math.princeton.edu/wp-content/uploads/annals-v160-n2-p12.pdf>.
- [2] A. O. L. Atkin and F. Morain. Elliptic curves and primality proving. *Mathematics of Computation*, 61(203):29–68, 1993.
- [3] R. Baillie and S. S. Wagstaff Jr. Lucas pseudoprimes. *Mathematics of Computation*, 35(152):1391–1417, 1980.
- [4] J. Brillhart, D. H. Lehmer, and J. L. Selfridge. New primality criteria and factorizations of $2^m \pm 1$. *Mathematics of Computation*, 29(130):620–647, 1975. <https://www.ams.org/journals/mcom/1975-29-130/S0025-5718-1975-0384673-1/S0025-5718-1975-0384673-1.pdf>.
- [5] H. Cohen and H. W. Lenstra Jr. Primality testing and Jacobi sums. *Mathematics of Computation*, 42(165):297–330, 1984. <https://pub.math.leidenuniv.nl/~lenstrahw/PUBLICATIONS/1984a/art.pdf>.
- [6] R. Crandall and C. Pomerance. *Prime Numbers: A Computational Perspective*. Springer, 2nd edition, 2005.
- [7] GNU MP. Prime testing algorithm, GMP manual. <https://gmplib.org/manual/Prime-Testing-Algorithm>.
- [8] F. Morain. The ECPP home page. <https://www.lix.polytechnique.fr/~morain/Prgms/ecpp.english.html>.
- [9] J. M. Pollard. Theorems on factorization and primality testing. *Mathematical Proceedings of the Cambridge Philosophical Society*, 76(3):521–528, 1974.
- [10] H. C. Pocklington. The determination of the prime or composite nature of large numbers by Fermat’s theorem. *Proceedings of the Cambridge Philosophical Society*, 18:29–30, 1914.
- [11] P. Pritchard. Explaining the wheel sieve. *Acta Informatica*, 17:477–485, 1982. <https://link.springer.com/content/pdf/10.1007/BF00264164.pdf>.
- [12] P. Pritchard. Linear prime-number sieves: a family tree. *Science of Computer Programming*, 9(1):17–35, 1987. <https://core.ac.uk/download/pdf/82210536.pdf>.
- [13] S. Ramanujan. On certain trigonometrical sums and their applications in the theory of numbers. *Transactions of the Cambridge Philosophical Society*, 22:259–276, 1918.
- [14] C. Runciman. Lazy wheel sieves and spirals of primes. *Journal of Functional Programming*, 7(2):219–225, 1997. <https://eprints.whiterose.ac.uk/id/eprint/3784/1/runcimanc1.pdf>.
- [15] L. Toth. Ramanujan sums. Survey notes. <https://ltoth.ttk.pte.hu/Ramanujan07.pdf>.